Task 4 — Per-Round LLM Iteration Trace (trial_gpt51)

Author: Zimo Ji

Per-round dump of the prompts I sent and the responses I received from `gpt-5.1` while iterating the test-driven code-generation experiment for `NewSubject.java`. Files reproduced verbatim from `reports/Task4_Data/trial_gpt51/{prompts,llm_outputs,test_results}/`.

Model: `gpt-5.1`, temperature 0.1, accessed via an OpenAI-compatible chat-completions endpoint.

Round-by-round summary

round	compile_ok	run	fail	pass_rate
1	True	62	1	98.4%
2	True	62	0	100.0%

Round 1

Test outcome


```
{
  "compile_ok": true,
  "run_count": 62,
  "fail_count": 1,
  "failures": [
    "test51(comp5111.assignment.cut.NewSubject_ESTest) :: arrays first differed at
element [0]; expected:<6> but was:<316>"
  ],
  "error": "",
  "raw_stdout": "10:41:14.887 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Seeing class for first time:
comp5111.assignment.cut.NewSubject_ESTest\n10:41:14.888 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Instrumenting class
'comp5111.assignment.cut.NewSubject_ESTest'.\n10:41:14.916 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Seeing class for first time:
comp5111.assignment.cut.NewSubject_ESTest_scaffolding\n10:41:14.916 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Instrumenting class
'comp5111.assignment.cut.NewSubject_ESTest_scaffolding'.\n10:41:14.918 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Keeping class:
comp5111.assignment.cut.NewSubject_ESTest_scaffolding\n10:41:14.918 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Keeping class:
comp5111.assignment.cut.NewSubject_ESTest\n10:41:15.097 [main] DEBUG
org.evosuite.runtime.sandbox.MSecurityManager - Adding privileged thread: \"Reference
Handler\"\n10:41:15.097 [main] DEBUG org.evosuite.runtime.sandbox.MSecurityManager -
Adding privileged thread: \"Finalizer\"\n10:41:15.097 [main] DEBUG
org.evosuite.runtime.sandbox.MSecurityManager - Adding privileged thread: \"Signal
Dispatcher\"\n10:41:15.097 [main] DEBUG org.evosuite.runtime.sandbox.MSecurityManager -
Adding privileged thread: \"Java2D Disposer\"\n10:41:15.097 [main] DEBUG
org.evosuite.runtime.sandbox.MSecurityManager - Adding privileged thread:
```

```
\"main\"\n10:41:15.097 [main] DEBUG org.evosuite.runtime.sandbox.MSecurityManager -
Adding privileged thread: \"Common-Cleaner\"\n10:41:15.115 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Seeing class for first time:
comp5111.assignment.cut.NewSubject\n10:41:15.115 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Instrumenting class
'comp5111.assignment.cut.NewSubject'.\n10:41:15.120 [main] INFO
org.evosuite.runtime.instrumentation.MethodCallReplacementClassAdapter - Adding mock
interface to class comp5111/assignment/cut/NewSubject\n10:41:15.191 [main] INFO
org.evosuite.runtime.instrumentation.CreateClassResetClassAdapter - Creating brand-new
static initializer in class comp5111/assignment/cut/NewSubject\n10:41:15.192 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Keeping class:
comp5111.assignment.cut.NewSubject\n10:41:15.207 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\n10:41:15.210 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\n10:41:15.213 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\n10:41:15.220 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\n10:41:15.227 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\n10:41:15.228 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\n10:41:15.230 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\n10:41:15.231 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\n10:41:15.232 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\n10:41:15.233 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\n10:41:15.235 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\n10:41:15.237 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\n10:41:15.238 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\n10:41:15.238 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\n10:41:15.239 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\n10:41:15.240 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new
thread\nRUN_COUNT=62\nFAIL_COUNT=1\nFAILURE>>>test51(comp5111.assignment.cut.NewSubject_EST
arrays first differed at element [0]; expected:<6> but was:<316><<<FAILURE\n",
  "raw_stderr": "WARNING: An illegal reflective access operation has occurred\nWARNING:
Illegal reflective access by org.evosuite.runtime.GuiSupport (file:/home/zimo/playground/
COMP5111-2026Spring-Assignments-Students/lib/evosuite-1.2.0.jar) to field
java.awt.GraphicsEnvironment.headless\nWARNING: Please consider reporting this to the
maintainers of org.evosuite.runtime.GuiSupport\nWARNING: Use --illegal-access=warn to
enable warnings of further illegal reflective access operations\nWARNING: All illegal
access operations will be denied in a future release\n"
}
```


Prompt sent to LLM (15138 chars)

```
You are a Java engineer. Implement the class `comp5111.assignment.cut.NewSubject` so that
every JUnit test in the provided test suite passes.

Rules:
 - Output exactly ONE fenced ```java code block containing the FULL contents of
NewSubject.java (package, imports if any, the entire class).
 - Do NOT modify the test suite. Do NOT add main(). Keep the package and class name
unchanged. Method signatures shown below are mandatory.
 - Output nothing outside the single code block.


=== Method signatures + Javadoc (mandatory shape) ===

package comp5111.assignment.cut;

/**
 * Task 4 reduced CUT.
 *
 * Ten public-static methods selected from {@link Subject}, with their
 * documented contracts and one private helper (isOneOf).  The patched
 * (correct) implementations from Task 2 are used so that EvoSuite's
```

```
 * generated tests pin down the *intended* behaviour, against which the
 * LLM's regenerated code is judged.
 *
 * Every signature is identical to the corresponding method in Subject.
 */
public class NewSubject {

    // ---- 1 ----
    /**
     * Returns true if str starts with prefix ignoring case, else false.
     * Returns false on any null argument or if str is shorter than prefix.
     */
    public static boolean startsWithIgnoreCase(String str, String prefix);

    // ---- 2 ----
    /**
     * Returns the prefix of str up to (but not including) the first
     * character that appears in {@code terminators}.  If no terminator
     * is found, the whole string is returned.
     */
    public static String parseToken(final String str, final char[] terminators);

    private static boolean isOneOf(char ch, final char[] charray);

    // ---- 3 ----
    /**
     * Returns the LAST contiguous run of digits in str interpreted as
     * an int. Empty/null/no-digit inputs return 0.
     * Examples: "1234" -> 1234, "a" -> 0, "1234a123" -> 123.
     */
    public static int extractIntInStr(String str);

    // ---- 4 ----
    /**
     * Parses a version string of form a, a.b, a.b.c, or a.b.c.d into a 4-int
     * array. Trailing components default to 0. More than 4 dot-segments or
     * malformed (consecutive dots) produces null.
     */
    public static int[] getVersionNo(final String versionString);

    // ---- 5 ----
    /**
     * Pads str on the left with padChar so the resulting length is at least
     * length. Null str is treated as "". If str is already long enough it is
     * returned unchanged.
     */
    public static String padLeft(String str, short length, char padChar);

    // ---- 6 ----
    /** Returns true iff year is a leap year (Gregorian rule). */
    public static boolean judgeLeapYear(int year);

    // ---- 7 ----
    /**
     * Returns the number of days in the given month (1-12) of the given
     * year. Returns -1 if month is out of range. February uses leap-year
     * rules.
     */
    public static int calcDaysInMonth(int year, int month);

    // ---- 8 ----
    /** Returns the calendar quarter (1-4) for month (1-12), or -1 if invalid. */
    public static int getQuarter(int month);
```

```
    // ---- 9 ----
    /**
     * Converts a 3-char month abbreviation (case-sensitive, e.g. "Jan",
     * "Sep") into its month number (1-12). Returns -1 on null, wrong
     * length, or unknown abbreviation.
     */
    public static int monAbbr2month(String abrr);

    // ---- 10 ----
    /**
     * Inverse of monAbbr2month: returns "Jan".."Dec" for month 1..12, and
     * "(invalid)" otherwise.
     */
    public static String month2MonAbbr(int month);
}


=== EvoSuite-generated test suite (passes against the ground truth) ===

/*
 * This file was automatically generated by EvoSuite
 * Thu May 07 11:32:05 GMT 2026
 */

package comp5111.assignment.cut;

import org.junit.Test;
import static org.junit.Assert.*;
import comp5111.assignment.cut.NewSubject;
import org.evosuite.runtime.EvoRunner;
import org.evosuite.runtime.EvoRunnerParameters;
import org.junit.runner.RunWith;

@RunWith(EvoRunner.class) @EvoRunnerParameters(mockJVMNonDeterminism = true, useVFS =
true, useVNET = true, resetStaticState = true, separateClassLoader = true)
public class NewSubject_ESTest extends NewSubject_ESTest_scaffolding {

  @Test(timeout = 4000)
  public void test00()  throws Throwable  {
      String string0 = NewSubject.month2MonAbbr(1493);
      assertEquals("(invalid)", string0);
      assertNotNull(string0);
  }

  @Test(timeout = 4000)
  public void test01()  throws Throwable  {
      String string0 = NewSubject.month2MonAbbr(12);
      assertEquals("Dec", string0);
      assertNotNull(string0);
  }

  @Test(timeout = 4000)
  public void test02()  throws Throwable  {
      String string0 = NewSubject.month2MonAbbr(11);
      assertEquals("Nov", string0);
      assertNotNull(string0);
  }

  @Test(timeout = 4000)
  public void test03()  throws Throwable  {
      String string0 = NewSubject.month2MonAbbr(10);
      assertEquals("Oct", string0);
      assertNotNull(string0);
  }
```

```
@Test(timeout = 4000)
public void test04()  throws Throwable  {
    String string0 = NewSubject.month2MonAbbr(9);
    assertEquals("Sep", string0);
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test05()  throws Throwable  {
    String string0 = NewSubject.month2MonAbbr(8);
    assertEquals("Aug", string0);
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test06()  throws Throwable  {
    String string0 = NewSubject.month2MonAbbr(7);
    assertEquals("Jul", string0);
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test07()  throws Throwable  {
    String string0 = NewSubject.month2MonAbbr(6);
    assertEquals("Jun", string0);
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test08()  throws Throwable  {
    String string0 = NewSubject.month2MonAbbr(5);
    assertEquals("May", string0);
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test09()  throws Throwable  {
    String string0 = NewSubject.month2MonAbbr(4);
    assertEquals("Apr", string0);
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test10()  throws Throwable  {
    String string0 = NewSubject.month2MonAbbr(3);
    assertEquals("Mar", string0);
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test11()  throws Throwable  {
    String string0 = NewSubject.month2MonAbbr(2);
    assertEquals("Feb", string0);
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test12()  throws Throwable  {
    String string0 = NewSubject.month2MonAbbr(1);
    assertEquals("Jan", string0);
    assertNotNull(string0);
}
```

```
    @Test(timeout = 4000)
    public void test13()  throws Throwable  {
        int int0 = NewSubject.monAbbr2month("Dec");
        assertEquals(12, int0);
    }

    @Test(timeout = 4000)
    public void test14()  throws Throwable  {
        int int0 = NewSubject.monAbbr2month("Nov");
        assertEquals(11, int0);
    }

    @Test(timeout = 4000)
    public void test15()  throws Throwable  {
        int int0 = NewSubject.monAbbr2month("Oct");
        assertEquals(10, int0);
    }

    @Test(timeout = 4000)
    public void test16()  throws Throwable  {
        int int0 = NewSubject.monAbbr2month("'Y~");
        assertEquals((-1), int0);
    }

    @Test(timeout = 4000)
    public void test17()  throws Throwable  {
        int int0 = NewSubject.monAbbr2month("Aug");
        assertEquals(8, int0);
    }

    @Test(timeout = 4000)
    public void test18()  throws Throwable  {
        int int0 = NewSubject.monAbbr2month("Jul");
        assertEquals(7, int0);
    }

    @Test(timeout = 4000)
    public void test19()  throws Throwable  {
        int int0 = NewSubject.monAbbr2month("Jun");
        assertEquals(6, int0);
    }

    @Test(timeout = 4000)
    public void test20()  throws Throwable  {
        int int0 = NewSubject.monAbbr2month("May");
        assertEquals(5, int0);
    }

    @Test(timeout = 4000)
    public void test21()  throws Throwable  {
        int int0 = NewSubject.monAbbr2month("Apr");
        assertEquals(4, int0);
    }

    @Test(timeout = 4000)
    public void test22()  throws Throwable  {
        int int0 = NewSubject.monAbbr2month("Mar");
        assertEquals(3, int0);
    }

    @Test(timeout = 4000)
    public void test23()  throws Throwable  {
        int int0 = NewSubject.monAbbr2month("Sep");
        assertEquals(9, int0);
```

```
}

@Test(timeout = 4000)
public void test24()  throws Throwable  {
    int int0 = NewSubject.monAbbr2month("Jan");
    assertEquals(1, int0);
}

@Test(timeout = 4000)
public void test25()  throws Throwable  {
    int int0 = NewSubject.monAbbr2month("(invalid)");
    assertEquals((-1), int0);
}

@Test(timeout = 4000)
public void test26()  throws Throwable  {
    int int0 = NewSubject.monAbbr2month("Feb");
    assertEquals(2, int0);
}

@Test(timeout = 4000)
public void test27()  throws Throwable  {
    int int0 = NewSubject.monAbbr2month((String) null);
    assertEquals((-1), int0);
}

@Test(timeout = 4000)
public void test28()  throws Throwable  {
    int int0 = NewSubject.getQuarter(11);
    assertEquals(4, int0);
}

@Test(timeout = 4000)
public void test29()  throws Throwable  {
    int int0 = NewSubject.getQuarter(4);
    assertEquals(2, int0);
}

@Test(timeout = 4000)
public void test30()  throws Throwable  {
    int int0 = NewSubject.getQuarter((short)8);
    assertEquals(3, int0);
}

@Test(timeout = 4000)
public void test31()  throws Throwable  {
    int int0 = NewSubject.getQuarter(946);
    assertEquals((-1), int0);
}

@Test(timeout = 4000)
public void test32()  throws Throwable  {
    int int0 = NewSubject.getQuarter(2);
    assertEquals(1, int0);
}

@Test(timeout = 4000)
public void test33()  throws Throwable  {
    int int0 = NewSubject.getQuarter((-1460));
    assertEquals((-1), int0);
}

@Test(timeout = 4000)
public void test34()  throws Throwable  {
```

```
    int int0 = NewSubject.calcDaysInMonth(11, 11);
    assertEquals(30, int0);
}

@Test(timeout = 4000)
public void test35()  throws Throwable  {
    int int0 = NewSubject.calcDaysInMonth((-1), 9);
    assertEquals(30, int0);
}

@Test(timeout = 4000)
public void test36()  throws Throwable  {
    int int0 = NewSubject.calcDaysInMonth(0, 6);
    assertEquals(30, int0);
}

@Test(timeout = 4000)
public void test37()  throws Throwable  {
    int int0 = NewSubject.calcDaysInMonth(2, 4);
    assertEquals(30, int0);
}

@Test(timeout = 4000)
public void test38()  throws Throwable  {
    int int0 = NewSubject.calcDaysInMonth((-1), 2);
    assertEquals(28, int0);
}

@Test(timeout = 4000)
public void test39()  throws Throwable  {
    int int0 = NewSubject.calcDaysInMonth(0, 3);
    assertEquals(31, int0);
}

@Test(timeout = 4000)
public void test40()  throws Throwable  {
    int int0 = NewSubject.calcDaysInMonth(1, 1726);
    assertEquals((-1), int0);
}

@Test(timeout = 4000)
public void test41()  throws Throwable  {
    int int0 = NewSubject.calcDaysInMonth((-3324), (-1570));
    assertEquals((-1), int0);
}

@Test(timeout = 4000)
public void test42()  throws Throwable  {
    boolean boolean0 = NewSubject.judgeLeapYear(3300);
    assertFalse(boolean0);
}

@Test(timeout = 4000)
public void test43()  throws Throwable  {
    int int0 = NewSubject.calcDaysInMonth((-3324), 2);
    assertEquals(29, int0);
}

@Test(timeout = 4000)
public void test44()  throws Throwable  {
    boolean boolean0 = NewSubject.judgeLeapYear(0);
    assertTrue(boolean0);
}
```

```
@Test(timeout = 4000)
public void test45()  throws Throwable  {
    String string0 = NewSubject.padLeft("Jun", (short)1741, 'G');
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test46()  throws Throwable  {
    String string0 = NewSubject.padLeft((String) null, (short) (-1), '(');
    assertEquals("", string0);
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test47()  throws Throwable  {
    int[] intArray0 = NewSubject.getVersionNo(".@:r~w'iW");
    assertNull(intArray0);
}

@Test(timeout = 4000)
public void test48()  throws Throwable  {
    int[] intArray0 = NewSubject.getVersionNo("oZ.>B6oZ.>B6oZ.>B6oZ.>B6");
    assertNull(intArray0);
}

@Test(timeout = 4000)
public void test49()  throws Throwable  {
    int[] intArray0 = NewSubject.getVersionNo("");
    assertNull(intArray0);
}

@Test(timeout = 4000)
public void test50()  throws Throwable  {
    int[] intArray0 = NewSubject.getVersionNo((String) null);
    assertNull(intArray0);
}

@Test(timeout = 4000)
public void test51()  throws Throwable  {
    int[] intArray0 = NewSubject.getVersionNo("$S3P&,*V+D&1{%U6");
    assertEquals(4, intArray0.length);
    assertNotNull(intArray0);
    assertArrayEquals(new int[] {6, 0, 0, 0}, intArray0);
}

@Test(timeout = 4000)
public void test52()  throws Throwable  {
    int int0 = NewSubject.extractIntInStr("");
    assertEquals(0, int0);
}

@Test(timeout = 4000)
public void test53()  throws Throwable  {
    int int0 = NewSubject.extractIntInStr((String) null);
    assertEquals(0, int0);
}

@Test(timeout = 4000)
public void test54()  throws Throwable  {
    char[] charArray0 = new char[5];
    charArray0[1] = 'A';
    String string0 = NewSubject.parseToken("Aug", charArray0);
    assertEquals("", string0);
    assertEquals(5, charArray0.length);
```

```
      assertNotNull(string0);
      assertArrayEquals(new char[] {'\u0000', 'A', '\u0000', '\u0000', '\u0000'},
charArray0);
  }

  @Test(timeout = 4000)
  public void test55()  throws Throwable  {
      char[] charArray0 = new char[6];
      String string0 = NewSubject.parseToken("(invalid)", charArray0);
      assertEquals("(invalid)", string0);
      assertEquals(6, charArray0.length);
      assertNotNull(string0);
      assertArrayEquals(new char[] {'\u0000', '\u0000', '\u0000', '\u0000', '\u0000',
'\u0000'}, charArray0);
  }

  @Test(timeout = 4000)
  public void test56()  throws Throwable  {
      boolean boolean0 = NewSubject.startsWithIgnoreCase("(invalid)", "Oct");
      assertFalse(boolean0);
  }

  @Test(timeout = 4000)
  public void test57()  throws Throwable  {
      boolean boolean0 = NewSubject.startsWithIgnoreCase("N", "Jun");
      assertFalse(boolean0);
  }

  @Test(timeout = 4000)
  public void test58()  throws Throwable  {
      boolean boolean0 = NewSubject.startsWithIgnoreCase("'Y~", (String) null);
      assertFalse(boolean0);
  }

  @Test(timeout = 4000)
  public void test59()  throws Throwable  {
      boolean boolean0 = NewSubject.startsWithIgnoreCase("$S3P&,*V+D&1{%U6",
"$S3P&,*V+D&1{%U6");
      assertTrue(boolean0);
  }

  @Test(timeout = 4000)
  public void test60()  throws Throwable  {
      boolean boolean0 = NewSubject.startsWithIgnoreCase((String) null, "Apr");
      assertFalse(boolean0);
  }

  @Test(timeout = 4000)
  public void test61()  throws Throwable  {
      NewSubject newSubject0 = new NewSubject();
      assertNotNull(newSubject0);
  }
}
```

Raw LLM response (8736 chars)

```
```java
package comp5111.assignment.cut;

/**
 * Task 4 reduced CUT.
```
```

```
 *
 * Ten public-static methods selected from {@link Subject}, with their
 * documented contracts and one private helper (isOneOf).  The patched
 * (correct) implementations from Task 2 are used so that EvoSuite's
 * generated tests pin down the *intended* behaviour, against which the
 * LLM's regenerated code is judged.
 *
 * Every signature is identical to the corresponding method in Subject.
 */
public class NewSubject {

    // ---- 1 ----
    /**
     * Returns true if str starts with prefix ignoring case, else false.
     * Returns false on any null argument or if str is shorter than prefix.
     */
    public static boolean startsWithIgnoreCase(String str, String prefix) {
        if (str == null || prefix == null) {
            return false;
        }
        if (str.length() < prefix.length()) {
            return false;
        }
        for (int i = 0; i < prefix.length(); i++) {
            char c1 = str.charAt(i);
            char c2 = prefix.charAt(i);
            if (Character.toLowerCase(c1) != Character.toLowerCase(c2)) {
                return false;
            }
        }
        return true;
    }

    // ---- 2 ----
    /**
     * Returns the prefix of str up to (but not including) the first
     * character that appears in {@code terminators}.  If no terminator
     * is found, the whole string is returned.
     */
    public static String parseToken(final String str, final char[] terminators) {
        if (str == null) {
            return null;
        }
        if (terminators == null || terminators.length == 0) {
            return str;
        }
        int len = str.length();
        for (int i = 0; i < len; i++) {
            char ch = str.charAt(i);
            if (isOneOf(ch, terminators)) {
                return str.substring(0, i);
            }
        }
        return str;
    }

    private static boolean isOneOf(char ch, final char[] charray) {
        if (charray == null) {
            return false;
        }
        for (char c : charray) {
            if (c == ch) {
                return true;
            }
```

```
        }
        return false;
    }

    // ---- 3 ----
    /**
     * Returns the LAST contiguous run of digits in str interpreted as
     * an int. Empty/null/no-digit inputs return 0.
     * Examples: "1234" -> 1234, "a" -> 0, "1234a123" -> 123.
     */
    public static int extractIntInStr(String str) {
        if (str == null || str.isEmpty()) {
            return 0;
        }
        int end = -1;
        int start = -1;
        for (int i = str.length() - 1; i >= 0; i--) {
            char c = str.charAt(i);
            if (Character.isDigit(c)) {
                if (end == -1) {
                    end = i;
                }
                start = i;
            } else if (end != -1) {
                break;
            }
        }
        if (end == -1) {
            return 0;
        }
        int result = 0;
        for (int i = start; i <= end; i++) {
            result = result * 10 + (str.charAt(i) - '0');
        }
        return result;
    }

    // ---- 4 ----
    /**
     * Parses a version string of form a, a.b, a.b.c, or a.b.c.d into a 4-int
     * array. Trailing components default to 0. More than 4 dot-segments or
     * malformed (consecutive dots) produces null.
     */
    public static int[] getVersionNo(final String versionString) {
        if (versionString == null || versionString.isEmpty()) {
            return null;
        }
        String[] parts = versionString.split("\\.", -1);
        if (parts.length == 0 || parts.length > 4) {
            return null;
        }
        int[] version = new int[] {0, 0, 0, 0};
        for (int i = 0; i < parts.length; i++) {
            String p = parts[i];
            if (p.isEmpty()) {
                // malformed: consecutive dots or leading/trailing dot
                return null;
            }
            int num = 0;
            boolean hasDigit = false;
            for (int j = 0; j < p.length(); j++) {
                char c = p.charAt(j);
                if (Character.isDigit(c)) {
                    hasDigit = true;
```

```
                    num = num * 10 + (c - '0');
                }
            }
            if (!hasDigit) {
                // no digit in this component is malformed
                return null;
            }
            if (i == 0) {
                // first component is most significant (a)
                version[0] = num;
            } else if (i == 1) {
                version[1] = num;
            } else if (i == 2) {
                version[2] = num;
            } else if (i == 3) {
                version[3] = num;
            }
        }
        return version;
    }

    // ---- 5 ----
    /**
     * Pads str on the left with padChar so the resulting length is at least
     * length. Null str is treated as "". If str is already long enough it is
     * returned unchanged.
     */
    public static String padLeft(String str, short length, char padChar) {
        if (str == null) {
            str = "";
        }
        int len = str.length();
        if (length <= len) {
            return str;
        }
        int pads = length - len;
        StringBuilder sb = new StringBuilder(length);
        for (int i = 0; i < pads; i++) {
            sb.append(padChar);
        }
        sb.append(str);
        return sb.toString();
    }

    // ---- 6 ----
    /** Returns true iff year is a leap year (Gregorian rule). */
    public static boolean judgeLeapYear(int year) {
        if (year % 400 == 0) {
            return true;
        }
        if (year % 100 == 0) {
            return false;
        }
        return year % 4 == 0;
    }

    // ---- 7 ----
    /**
     * Returns the number of days in the given month (1-12) of the given
     * year. Returns -1 if month is out of range. February uses leap-year
     * rules.
     */
    public static int calcDaysInMonth(int year, int month) {
        if (month < 1 || month > 12) {
```

```
            return -1;
        }
        switch (month) {
            case 1:
            case 3:
            case 5:
            case 7:
            case 8:
            case 10:
            case 12:
                return 31;
            case 4:
            case 6:
            case 9:
            case 11:
                return 30;
            case 2:
                return judgeLeapYear(year) ? 29 : 28;
            default:
                return -1;
        }
    }

    // ---- 8 ----
    /** Returns the calendar quarter (1-4) for month (1-12), or -1 if invalid. */
    public static int getQuarter(int month) {
        if (month < 1 || month > 12) {
            return -1;
        }
        return (month - 1) / 3 + 1;
    }

    // ---- 9 ----
    /**
     * Converts a 3-char month abbreviation (case-sensitive, e.g. "Jan",
     * "Sep") into its month number (1-12). Returns -1 on null, wrong
     * length, or unknown abbreviation.
     */
    public static int monAbbr2month(String abrr) {
        if (abrr == null || abrr.length() != 3) {
            return -1;
        }
        switch (abrr) {
            case "Jan":
                return 1;
            case "Feb":
                return 2;
            case "Mar":
                return 3;
            case "Apr":
                return 4;
            case "May":
                return 5;
            case "Jun":
                return 6;
            case "Jul":
                return 7;
            case "Aug":
                return 8;
            case "Sep":
                return 9;
            case "Oct":
                return 10;
            case "Nov":
```

```
                return 11;
            case "Dec":
                return 12;
            default:
                return -1;
        }
    }

    // ---- 10 ----
    /**
     * Inverse of monAbbr2month: returns "Jan".."Dec" for month 1..12, and
     * "(invalid)" otherwise.
     */
    public static String month2MonAbbr(int month) {
        switch (month) {
            case 1:
                return "Jan";
            case 2:
                return "Feb";
            case 3:
                return "Mar";
            case 4:
                return "Apr";
            case 5:
                return "May";
            case 6:
                return "Jun";
            case 7:
                return "Jul";
            case 8:
                return "Aug";
            case 9:
                return "Sep";
            case 10:
                return "Oct";
            case 11:
                return "Nov";
            case 12:
                return "Dec";
            default:
                return "(invalid)";
        }
    }
}
```

Extracted NewSubject.java for this round

```java
package comp5111.assignment.cut;

/**
 * Task 4 reduced CUT.
 *
 * Ten public-static methods selected from {@link Subject}, with their
 * documented contracts and one private helper (isOneOf).  The patched
 * (correct) implementations from Task 2 are used so that EvoSuite's
 * generated tests pin down the *intended* behaviour, against which the
 * LLM's regenerated code is judged.
 *
 * Every signature is identical to the corresponding method in Subject.
 */
public class NewSubject {
```

```
// ---- 1 ----
/**
 * Returns true if str starts with prefix ignoring case, else false.
 * Returns false on any null argument or if str is shorter than prefix.
 */
public static boolean startsWithIgnoreCase(String str, String prefix) {
    if (str == null || prefix == null) {
        return false;
    }
    if (str.length() < prefix.length()) {
        return false;
    }
    for (int i = 0; i < prefix.length(); i++) {
        char c1 = str.charAt(i);
        char c2 = prefix.charAt(i);
        if (Character.toLowerCase(c1) != Character.toLowerCase(c2)) {
            return false;
        }
    }
    return true;
}

// ---- 2 ----
/**
 * Returns the prefix of str up to (but not including) the first
 * character that appears in {@code terminators}.  If no terminator
 * is found, the whole string is returned.
 */
public static String parseToken(final String str, final char[] terminators) {
    if (str == null) {
        return null;
    }
    if (terminators == null || terminators.length == 0) {
        return str;
    }
    int len = str.length();
    for (int i = 0; i < len; i++) {
        char ch = str.charAt(i);
        if (isOneOf(ch, terminators)) {
            return str.substring(0, i);
        }
    }
    return str;
}

private static boolean isOneOf(char ch, final char[] charray) {
    if (charray == null) {
        return false;
    }
    for (char c : charray) {
        if (c == ch) {
            return true;
        }
    }
    return false;
}

// ---- 3 ----
/**
 * Returns the LAST contiguous run of digits in str interpreted as
 * an int. Empty/null/no-digit inputs return 0.
 * Examples: "1234" -> 1234, "a" -> 0, "1234a123" -> 123.
 */
```

```
    public static int extractIntInStr(String str) {
        if (str == null || str.isEmpty()) {
            return 0;
        }
        int end = -1;
        int start = -1;
        for (int i = str.length() - 1; i >= 0; i--) {
            char c = str.charAt(i);
            if (Character.isDigit(c)) {
                if (end == -1) {
                    end = i;
                }
                start = i;
            } else if (end != -1) {
                break;
            }
        }
        if (end == -1) {
            return 0;
        }
        int result = 0;
        for (int i = start; i <= end; i++) {
            result = result * 10 + (str.charAt(i) - '0');
        }
        return result;
    }

    // ---- 4 ----
    /**
     * Parses a version string of form a, a.b, a.b.c, or a.b.c.d into a 4-int
     * array. Trailing components default to 0. More than 4 dot-segments or
     * malformed (consecutive dots) produces null.
     */
    public static int[] getVersionNo(final String versionString) {
        if (versionString == null || versionString.isEmpty()) {
            return null;
        }
        String[] parts = versionString.split("\\.", -1);
        if (parts.length == 0 || parts.length > 4) {
            return null;
        }
        int[] version = new int[] {0, 0, 0, 0};
        for (int i = 0; i < parts.length; i++) {
            String p = parts[i];
            if (p.isEmpty()) {
                // malformed: consecutive dots or leading/trailing dot
                return null;
            }
            int num = 0;
            boolean hasDigit = false;
            for (int j = 0; j < p.length(); j++) {
                char c = p.charAt(j);
                if (Character.isDigit(c)) {
                    hasDigit = true;
                    num = num * 10 + (c - '0');
                }
            }
            if (!hasDigit) {
                // no digit in this component is malformed
                return null;
            }
            if (i == 0) {
                // first component is most significant (a)
                version[0] = num;
```

```
            } else if (i == 1) {
                version[1] = num;
            } else if (i == 2) {
                version[2] = num;
            } else if (i == 3) {
                version[3] = num;
            }
        }
        return version;
    }

    // ---- 5 ----
    /**
     * Pads str on the left with padChar so the resulting length is at least
     * length. Null str is treated as "". If str is already long enough it is
     * returned unchanged.
     */
    public static String padLeft(String str, short length, char padChar) {
        if (str == null) {
            str = "";
        }
        int len = str.length();
        if (length <= len) {
            return str;
        }
        int pads = length - len;
        StringBuilder sb = new StringBuilder(length);
        for (int i = 0; i < pads; i++) {
            sb.append(padChar);
        }
        sb.append(str);
        return sb.toString();
    }

    // ---- 6 ----
    /** Returns true iff year is a leap year (Gregorian rule). */
    public static boolean judgeLeapYear(int year) {
        if (year % 400 == 0) {
            return true;
        }
        if (year % 100 == 0) {
            return false;
        }
        return year % 4 == 0;
    }

    // ---- 7 ----
    /**
     * Returns the number of days in the given month (1-12) of the given
     * year. Returns -1 if month is out of range. February uses leap-year
     * rules.
     */
    public static int calcDaysInMonth(int year, int month) {
        if (month < 1 || month > 12) {
            return -1;
        }
        switch (month) {
            case 1:
            case 3:
            case 5:
            case 7:
            case 8:
            case 10:
            case 12:
```

```
                return 31;
            case 4:
            case 6:
            case 9:
            case 11:
                return 30;
            case 2:
                return judgeLeapYear(year) ? 29 : 28;
            default:
                return -1;
        }
    }

    // ---- 8 ----
    /** Returns the calendar quarter (1-4) for month (1-12), or -1 if invalid. */
    public static int getQuarter(int month) {
        if (month < 1 || month > 12) {
            return -1;
        }
        return (month - 1) / 3 + 1;
    }

    // ---- 9 ----
    /**
     * Converts a 3-char month abbreviation (case-sensitive, e.g. "Jan",
     * "Sep") into its month number (1-12). Returns -1 on null, wrong
     * length, or unknown abbreviation.
     */
    public static int monAbbr2month(String abrr) {
        if (abrr == null || abrr.length() != 3) {
            return -1;
        }
        switch (abrr) {
            case "Jan":
                return 1;
            case "Feb":
                return 2;
            case "Mar":
                return 3;
            case "Apr":
                return 4;
            case "May":
                return 5;
            case "Jun":
                return 6;
            case "Jul":
                return 7;
            case "Aug":
                return 8;
            case "Sep":
                return 9;
            case "Oct":
                return 10;
            case "Nov":
                return 11;
            case "Dec":
                return 12;
            default:
                return -1;
        }
    }

    // ---- 10 ----
    /**
```

```
     * Inverse of monAbbr2month: returns "Jan".."Dec" for month 1..12, and
     * "(invalid)" otherwise.
     */
    public static String month2MonAbbr(int month) {
        switch (month) {
            case 1:
                return "Jan";
            case 2:
                return "Feb";
            case 3:
                return "Mar";
            case 4:
                return "Apr";
            case 5:
                return "May";
            case 6:
                return "Jun";
            case 7:
                return "Jul";
            case 8:
                return "Aug";
            case 9:
                return "Sep";
            case 10:
                return "Oct";
            case 11:
                return "Nov";
            case 12:
                return "Dec";
            default:
                return "(invalid)";
        }
    }
}
```

Round 2

Test outcome


```
{
  "compile_ok": true,
  "run_count": 62,
  "fail_count": 0,
  "failures": [],
  "error": "",
  "raw_stdout": "10:41:27.372 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Seeing class for first time:
comp5111.assignment.cut.NewSubject_ESTest\n10:41:27.373 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Instrumenting class
'comp5111.assignment.cut.NewSubject_ESTest'.\n10:41:27.401 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Seeing class for first time:
comp5111.assignment.cut.NewSubject_ESTest_scaffolding\n10:41:27.401 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Instrumenting class
'comp5111.assignment.cut.NewSubject_ESTest_scaffolding'.\n10:41:27.403 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Keeping class:
comp5111.assignment.cut.NewSubject_ESTest_scaffolding\n10:41:27.403 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Keeping class:
comp5111.assignment.cut.NewSubject_ESTest\n10:41:27.579 [main] DEBUG
org.evosuite.runtime.sandbox.MSecurityManager - Adding privileged thread: \"Reference
```

```
Handler\"\n10:41:27.579 [main] DEBUG org.evosuite.runtime.sandbox.MSecurityManager -
Adding privileged thread: \"Finalizer\"\n10:41:27.579 [main] DEBUG
org.evosuite.runtime.sandbox.MSecurityManager - Adding privileged thread: \"Signal
Dispatcher\"\n10:41:27.579 [main] DEBUG org.evosuite.runtime.sandbox.MSecurityManager -
Adding privileged thread: \"Java2D Disposer\"\n10:41:27.579 [main] DEBUG
org.evosuite.runtime.sandbox.MSecurityManager - Adding privileged thread:
\"main\"\n10:41:27.579 [main] DEBUG org.evosuite.runtime.sandbox.MSecurityManager -
Adding privileged thread: \"Common-Cleaner\"\n10:41:27.597 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Seeing class for first time:
comp5111.assignment.cut.NewSubject\n10:41:27.597 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Instrumenting class
'comp5111.assignment.cut.NewSubject'.\n10:41:27.602 [main] INFO
org.evosuite.runtime.instrumentation.MethodCallReplacementClassAdapter - Adding mock
interface to class comp5111/assignment/cut/NewSubject\n10:41:27.673 [main] INFO
org.evosuite.runtime.instrumentation.CreateClassResetClassAdapter - Creating brand-new
static initializer in class comp5111/assignment/cut/NewSubject\n10:41:27.674 [main] INFO
org.evosuite.runtime.instrumentation.EvoClassLoader - Keeping class:
comp5111.assignment.cut.NewSubject\n10:41:27.685 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new thread\n10:41:27.712 [main] INFO
org.evosuite.runtime.thread.ThreadStopper - Found new
thread\nRUN_COUNT=62\nFAIL_COUNT=0\n",
  "raw_stderr": "WARNING: An illegal reflective access operation has occurred\nWARNING:
Illegal reflective access by org.evosuite.runtime.GuiSupport (file:/home/zimo/playground/
COMP5111-2026Spring-Assignments-Students/lib/evosuite-1.2.0.jar) to field
java.awt.GraphicsEnvironment.headless\nWARNING: Please consider reporting this to the
maintainers of org.evosuite.runtime.GuiSupport\nWARNING: Use --illegal-access=warn to
enable warnings of further illegal reflective access operations\nWARNING: All illegal
access operations will be denied in a future release\n"
}
```


Prompt sent to LLM (24231 chars)

```
You are a Java engineer. Implement the class `comp5111.assignment.cut.NewSubject` so that
every JUnit test in the provided test suite passes.

Rules:
 - Output exactly ONE fenced ```java code block containing the FULL contents of
NewSubject.java (package, imports if any, the entire class).
 - Do NOT modify the test suite. Do NOT add main(). Keep the package and class name
unchanged. Method signatures shown below are mandatory.
 - Output nothing outside the single code block.


=== Method signatures + Javadoc (mandatory shape) ===

package comp5111.assignment.cut;

/**
 * Task 4 reduced CUT.
 *
 * Ten public-static methods selected from {@link Subject}, with their
 * documented contracts and one private helper (isOneOf).  The patched
 * (correct) implementations from Task 2 are used so that EvoSuite's
 * generated tests pin down the *intended* behaviour, against which the
 * LLM's regenerated code is judged.
 *
 * Every signature is identical to the corresponding method in Subject.
 */
public class NewSubject {

    // ---- 1 ----
    /**
```

```
     * Returns true if str starts with prefix ignoring case, else false.
     * Returns false on any null argument or if str is shorter than prefix.
     */
    public static boolean startsWithIgnoreCase(String str, String prefix);

    // ---- 2 ----
    /**
     * Returns the prefix of str up to (but not including) the first
     * character that appears in {@code terminators}.  If no terminator
     * is found, the whole string is returned.
     */
    public static String parseToken(final String str, final char[] terminators);

    private static boolean isOneOf(char ch, final char[] charray);

    // ---- 3 ----
    /**
     * Returns the LAST contiguous run of digits in str interpreted as
     * an int. Empty/null/no-digit inputs return 0.
     * Examples: "1234" -> 1234, "a" -> 0, "1234a123" -> 123.
     */
    public static int extractIntInStr(String str);

    // ---- 4 ----
    /**
     * Parses a version string of form a, a.b, a.b.c, or a.b.c.d into a 4-int
     * array. Trailing components default to 0. More than 4 dot-segments or
     * malformed (consecutive dots) produces null.
     */
    public static int[] getVersionNo(final String versionString);

    // ---- 5 ----
    /**
     * Pads str on the left with padChar so the resulting length is at least
     * length. Null str is treated as "". If str is already long enough it is
     * returned unchanged.
     */
    public static String padLeft(String str, short length, char padChar);

    // ---- 6 ----
    /** Returns true iff year is a leap year (Gregorian rule). */
    public static boolean judgeLeapYear(int year);

    // ---- 7 ----
    /**
     * Returns the number of days in the given month (1-12) of the given
     * year. Returns -1 if month is out of range. February uses leap-year
     * rules.
     */
    public static int calcDaysInMonth(int year, int month);

    // ---- 8 ----
    /** Returns the calendar quarter (1-4) for month (1-12), or -1 if invalid. */
    public static int getQuarter(int month);

    // ---- 9 ----
    /**
     * Converts a 3-char month abbreviation (case-sensitive, e.g. "Jan",
     * "Sep") into its month number (1-12). Returns -1 on null, wrong
     * length, or unknown abbreviation.
     */
    public static int monAbbr2month(String abrr);

    // ---- 10 ----
```

```
    /**
     * Inverse of monAbbr2month: returns "Jan".."Dec" for month 1..12, and
     * "(invalid)" otherwise.
     */
    public static String month2MonAbbr(int month);
}


=== EvoSuite-generated test suite (passes against the ground truth) ===

/*
 * This file was automatically generated by EvoSuite
 * Thu May 07 11:32:05 GMT 2026
 */

package comp5111.assignment.cut;

import org.junit.Test;
import static org.junit.Assert.*;
import comp5111.assignment.cut.NewSubject;
import org.evosuite.runtime.EvoRunner;
import org.evosuite.runtime.EvoRunnerParameters;
import org.junit.runner.RunWith;

@RunWith(EvoRunner.class) @EvoRunnerParameters(mockJVMNonDeterminism = true, useVFS =
true, useVNET = true, resetStaticState = true, separateClassLoader = true)
public class NewSubject_ESTest extends NewSubject_ESTest_scaffolding {

  @Test(timeout = 4000)
  public void test00()  throws Throwable  {
      String string0 = NewSubject.month2MonAbbr(1493);
      assertEquals("(invalid)", string0);
      assertNotNull(string0);
  }

  @Test(timeout = 4000)
  public void test01()  throws Throwable  {
      String string0 = NewSubject.month2MonAbbr(12);
      assertEquals("Dec", string0);
      assertNotNull(string0);
  }

  @Test(timeout = 4000)
  public void test02()  throws Throwable  {
      String string0 = NewSubject.month2MonAbbr(11);
      assertEquals("Nov", string0);
      assertNotNull(string0);
  }

  @Test(timeout = 4000)
  public void test03()  throws Throwable  {
      String string0 = NewSubject.month2MonAbbr(10);
      assertEquals("Oct", string0);
      assertNotNull(string0);
  }

  @Test(timeout = 4000)
  public void test04()  throws Throwable  {
      String string0 = NewSubject.month2MonAbbr(9);
      assertEquals("Sep", string0);
      assertNotNull(string0);
  }

  @Test(timeout = 4000)
```

```
public void test05()  throws Throwable  {
    String string0 = NewSubject.month2MonAbbr(8);
    assertEquals("Aug", string0);
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test06()  throws Throwable  {
    String string0 = NewSubject.month2MonAbbr(7);
    assertEquals("Jul", string0);
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test07()  throws Throwable  {
    String string0 = NewSubject.month2MonAbbr(6);
    assertEquals("Jun", string0);
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test08()  throws Throwable  {
    String string0 = NewSubject.month2MonAbbr(5);
    assertEquals("May", string0);
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test09()  throws Throwable  {
    String string0 = NewSubject.month2MonAbbr(4);
    assertEquals("Apr", string0);
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test10()  throws Throwable  {
    String string0 = NewSubject.month2MonAbbr(3);
    assertEquals("Mar", string0);
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test11()  throws Throwable  {
    String string0 = NewSubject.month2MonAbbr(2);
    assertEquals("Feb", string0);
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test12()  throws Throwable  {
    String string0 = NewSubject.month2MonAbbr(1);
    assertEquals("Jan", string0);
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test13()  throws Throwable  {
    int int0 = NewSubject.monAbbr2month("Dec");
    assertEquals(12, int0);
}

@Test(timeout = 4000)
public void test14()  throws Throwable  {
    int int0 = NewSubject.monAbbr2month("Nov");
```

```
    assertEquals(11, int0);
}

@Test(timeout = 4000)
public void test15()  throws Throwable  {
    int int0 = NewSubject.monAbbr2month("Oct");
    assertEquals(10, int0);
}

@Test(timeout = 4000)
public void test16()  throws Throwable  {
    int int0 = NewSubject.monAbbr2month("'Y~");
    assertEquals((-1), int0);
}

@Test(timeout = 4000)
public void test17()  throws Throwable  {
    int int0 = NewSubject.monAbbr2month("Aug");
    assertEquals(8, int0);
}

@Test(timeout = 4000)
public void test18()  throws Throwable  {
    int int0 = NewSubject.monAbbr2month("Jul");
    assertEquals(7, int0);
}

@Test(timeout = 4000)
public void test19()  throws Throwable  {
    int int0 = NewSubject.monAbbr2month("Jun");
    assertEquals(6, int0);
}

@Test(timeout = 4000)
public void test20()  throws Throwable  {
    int int0 = NewSubject.monAbbr2month("May");
    assertEquals(5, int0);
}

@Test(timeout = 4000)
public void test21()  throws Throwable  {
    int int0 = NewSubject.monAbbr2month("Apr");
    assertEquals(4, int0);
}

@Test(timeout = 4000)
public void test22()  throws Throwable  {
    int int0 = NewSubject.monAbbr2month("Mar");
    assertEquals(3, int0);
}

@Test(timeout = 4000)
public void test23()  throws Throwable  {
    int int0 = NewSubject.monAbbr2month("Sep");
    assertEquals(9, int0);
}

@Test(timeout = 4000)
public void test24()  throws Throwable  {
    int int0 = NewSubject.monAbbr2month("Jan");
    assertEquals(1, int0);
}

@Test(timeout = 4000)
```

```
public void test25()  throws Throwable  {
    int int0 = NewSubject.monAbbr2month("(invalid)");
    assertEquals((-1), int0);
}

@Test(timeout = 4000)
public void test26()  throws Throwable  {
    int int0 = NewSubject.monAbbr2month("Feb");
    assertEquals(2, int0);
}

@Test(timeout = 4000)
public void test27()  throws Throwable  {
    int int0 = NewSubject.monAbbr2month((String) null);
    assertEquals((-1), int0);
}

@Test(timeout = 4000)
public void test28()  throws Throwable  {
    int int0 = NewSubject.getQuarter(11);
    assertEquals(4, int0);
}

@Test(timeout = 4000)
public void test29()  throws Throwable  {
    int int0 = NewSubject.getQuarter(4);
    assertEquals(2, int0);
}

@Test(timeout = 4000)
public void test30()  throws Throwable  {
    int int0 = NewSubject.getQuarter((short)8);
    assertEquals(3, int0);
}

@Test(timeout = 4000)
public void test31()  throws Throwable  {
    int int0 = NewSubject.getQuarter(946);
    assertEquals((-1), int0);
}

@Test(timeout = 4000)
public void test32()  throws Throwable  {
    int int0 = NewSubject.getQuarter(2);
    assertEquals(1, int0);
}

@Test(timeout = 4000)
public void test33()  throws Throwable  {
    int int0 = NewSubject.getQuarter((-1460));
    assertEquals((-1), int0);
}

@Test(timeout = 4000)
public void test34()  throws Throwable  {
    int int0 = NewSubject.calcDaysInMonth(11, 11);
    assertEquals(30, int0);
}

@Test(timeout = 4000)
public void test35()  throws Throwable  {
    int int0 = NewSubject.calcDaysInMonth((-1), 9);
    assertEquals(30, int0);
}
```

```
@Test(timeout = 4000)
public void test36()  throws Throwable  {
    int int0 = NewSubject.calcDaysInMonth(0, 6);
    assertEquals(30, int0);
}

@Test(timeout = 4000)
public void test37()  throws Throwable  {
    int int0 = NewSubject.calcDaysInMonth(2, 4);
    assertEquals(30, int0);
}

@Test(timeout = 4000)
public void test38()  throws Throwable  {
    int int0 = NewSubject.calcDaysInMonth((-1), 2);
    assertEquals(28, int0);
}

@Test(timeout = 4000)
public void test39()  throws Throwable  {
    int int0 = NewSubject.calcDaysInMonth(0, 3);
    assertEquals(31, int0);
}

@Test(timeout = 4000)
public void test40()  throws Throwable  {
    int int0 = NewSubject.calcDaysInMonth(1, 1726);
    assertEquals((-1), int0);
}

@Test(timeout = 4000)
public void test41()  throws Throwable  {
    int int0 = NewSubject.calcDaysInMonth((-3324), (-1570));
    assertEquals((-1), int0);
}

@Test(timeout = 4000)
public void test42()  throws Throwable  {
    boolean boolean0 = NewSubject.judgeLeapYear(3300);
    assertFalse(boolean0);
}

@Test(timeout = 4000)
public void test43()  throws Throwable  {
    int int0 = NewSubject.calcDaysInMonth((-3324), 2);
    assertEquals(29, int0);
}

@Test(timeout = 4000)
public void test44()  throws Throwable  {
    boolean boolean0 = NewSubject.judgeLeapYear(0);
    assertTrue(boolean0);
}

@Test(timeout = 4000)
public void test45()  throws Throwable  {
    String string0 = NewSubject.padLeft("Jun", (short)1741, 'G');
    assertNotNull(string0);
}

@Test(timeout = 4000)
public void test46()  throws Throwable  {
    String string0 = NewSubject.padLeft((String) null, (short) (-1), '(');
```

```
      assertEquals("", string0);
      assertNotNull(string0);
  }

  @Test(timeout = 4000)
  public void test47()  throws Throwable  {
      int[] intArray0 = NewSubject.getVersionNo(".@:r~w'iW");
      assertNull(intArray0);
  }

  @Test(timeout = 4000)
  public void test48()  throws Throwable  {
      int[] intArray0 = NewSubject.getVersionNo("oZ.>B6oZ.>B6oZ.>B6oZ.>B6");
      assertNull(intArray0);
  }

  @Test(timeout = 4000)
  public void test49()  throws Throwable  {
      int[] intArray0 = NewSubject.getVersionNo("");
      assertNull(intArray0);
  }

  @Test(timeout = 4000)
  public void test50()  throws Throwable  {
      int[] intArray0 = NewSubject.getVersionNo((String) null);
      assertNull(intArray0);
  }

  @Test(timeout = 4000)
  public void test51()  throws Throwable  {
      int[] intArray0 = NewSubject.getVersionNo("$S3P&,*V+D&1{%U6");
      assertEquals(4, intArray0.length);
      assertNotNull(intArray0);
      assertArrayEquals(new int[] {6, 0, 0, 0}, intArray0);
  }

  @Test(timeout = 4000)
  public void test52()  throws Throwable  {
      int int0 = NewSubject.extractIntInStr("");
      assertEquals(0, int0);
  }

  @Test(timeout = 4000)
  public void test53()  throws Throwable  {
      int int0 = NewSubject.extractIntInStr((String) null);
      assertEquals(0, int0);
  }

  @Test(timeout = 4000)
  public void test54()  throws Throwable  {
      char[] charArray0 = new char[5];
      charArray0[1] = 'A';
      String string0 = NewSubject.parseToken("Aug", charArray0);
      assertEquals("", string0);
      assertEquals(5, charArray0.length);
      assertNotNull(string0);
      assertArrayEquals(new char[] {'\u0000', 'A', '\u0000', '\u0000', '\u0000'},
charArray0);
  }

  @Test(timeout = 4000)
  public void test55()  throws Throwable  {
      char[] charArray0 = new char[6];
      String string0 = NewSubject.parseToken("(invalid)", charArray0);
```

```
      assertEquals("(invalid)", string0);
      assertEquals(6, charArray0.length);
      assertNotNull(string0);
      assertArrayEquals(new char[] {'\u0000', '\u0000', '\u0000', '\u0000', '\u0000',
'\u0000'}, charArray0);
  }

  @Test(timeout = 4000)
  public void test56()  throws Throwable  {
      boolean boolean0 = NewSubject.startsWithIgnoreCase("(invalid)", "Oct");
      assertFalse(boolean0);
  }

  @Test(timeout = 4000)
  public void test57()  throws Throwable  {
      boolean boolean0 = NewSubject.startsWithIgnoreCase("N", "Jun");
      assertFalse(boolean0);
  }

  @Test(timeout = 4000)
  public void test58()  throws Throwable  {
      boolean boolean0 = NewSubject.startsWithIgnoreCase("'Y~", (String) null);
      assertFalse(boolean0);
  }

  @Test(timeout = 4000)
  public void test59()  throws Throwable  {
      boolean boolean0 = NewSubject.startsWithIgnoreCase("$S3P&,*V+D&1{%U6",
"$S3P&,*V+D&1{%U6");
      assertTrue(boolean0);
  }

  @Test(timeout = 4000)
  public void test60()  throws Throwable  {
      boolean boolean0 = NewSubject.startsWithIgnoreCase((String) null, "Apr");
      assertFalse(boolean0);
  }

  @Test(timeout = 4000)
  public void test61()  throws Throwable  {
      NewSubject newSubject0 = new NewSubject();
      assertNotNull(newSubject0);
  }
}


=== Your previous implementation (round 1) ===

package comp5111.assignment.cut;

/**
 * Task 4 reduced CUT.
 *
 * Ten public-static methods selected from {@link Subject}, with their
 * documented contracts and one private helper (isOneOf).  The patched
 * (correct) implementations from Task 2 are used so that EvoSuite's
 * generated tests pin down the *intended* behaviour, against which the
 * LLM's regenerated code is judged.
 *
 * Every signature is identical to the corresponding method in Subject.
 */
public class NewSubject {

    // ---- 1 ----
```

```
    /**
     * Returns true if str starts with prefix ignoring case, else false.
     * Returns false on any null argument or if str is shorter than prefix.
     */
    public static boolean startsWithIgnoreCase(String str, String prefix) {
        if (str == null || prefix == null) {
            return false;
        }
        if (str.length() < prefix.length()) {
            return false;
        }
        for (int i = 0; i < prefix.length(); i++) {
            char c1 = str.charAt(i);
            char c2 = prefix.charAt(i);
            if (Character.toLowerCase(c1) != Character.toLowerCase(c2)) {
                return false;
            }
        }
        return true;
    }

    // ---- 2 ----
    /**
     * Returns the prefix of str up to (but not including) the first
     * character that appears in {@code terminators}.  If no terminator
     * is found, the whole string is returned.
     */
    public static String parseToken(final String str, final char[] terminators) {
        if (str == null) {
            return null;
        }
        if (terminators == null || terminators.length == 0) {
            return str;
        }
        int len = str.length();
        for (int i = 0; i < len; i++) {
            char ch = str.charAt(i);
            if (isOneOf(ch, terminators)) {
                return str.substring(0, i);
            }
        }
        return str;
    }

    private static boolean isOneOf(char ch, final char[] charray) {
        if (charray == null) {
            return false;
        }
        for (char c : charray) {
            if (c == ch) {
                return true;
            }
        }
        return false;
    }

    // ---- 3 ----
    /**
     * Returns the LAST contiguous run of digits in str interpreted as
     * an int. Empty/null/no-digit inputs return 0.
     * Examples: "1234" -> 1234, "a" -> 0, "1234a123" -> 123.
     */
    public static int extractIntInStr(String str) {
        if (str == null || str.isEmpty()) {
```

```
            return 0;
        }
        int end = -1;
        int start = -1;
        for (int i = str.length() - 1; i >= 0; i--) {
            char c = str.charAt(i);
            if (Character.isDigit(c)) {
                if (end == -1) {
                    end = i;
                }
                start = i;
            } else if (end != -1) {
                break;
            }
        }
        if (end == -1) {
            return 0;
        }
        int result = 0;
        for (int i = start; i <= end; i++) {
            result = result * 10 + (str.charAt(i) - '0');
        }
        return result;
    }

    // ---- 4 ----
    /**
     * Parses a version string of form a, a.b, a.b.c, or a.b.c.d into a 4-int
     * array. Trailing components default to 0. More than 4 dot-segments or
     * malformed (consecutive dots) produces null.
     */
    public static int[] getVersionNo(final String versionString) {
        if (versionString == null || versionString.isEmpty()) {
            return null;
        }
        String[] parts = versionString.split("\\.", -1);
        if (parts.length == 0 || parts.length > 4) {
            return null;
        }
        int[] version = new int[] {0, 0, 0, 0};
        for (int i = 0; i < parts.length; i++) {
            String p = parts[i];
            if (p.isEmpty()) {
                // malformed: consecutive dots or leading/trailing dot
                return null;
            }
            int num = 0;
            boolean hasDigit = false;
            for (int j = 0; j < p.length(); j++) {
                char c = p.charAt(j);
                if (Character.isDigit(c)) {
                    hasDigit = true;
                    num = num * 10 + (c - '0');
                }
            }
            if (!hasDigit) {
                // no digit in this component is malformed
                return null;
            }
            if (i == 0) {
                // first component is most significant (a)
                version[0] = num;
            } else if (i == 1) {
                version[1] = num;
```

```
            } else if (i == 2) {
                version[2] = num;
            } else if (i == 3) {
                version[3] = num;
            }
        }
        return version;
    }

    // ---- 5 ----
    /**
     * Pads str on the left with padChar so the resulting length is at least
     * length. Null str is treated as "". If str is already long enough it is
     * returned unchanged.
     */
    public static String padLeft(String str, short length, char padChar) {
        if (str == null) {
            str = "";
        }
        int len = str.length();
        if (length <= len) {
            return str;
        }
        int pads = length - len;
        StringBuilder sb = new StringBuilder(length);
        for (int i = 0; i < pads; i++) {
            sb.append(padChar);
        }
        sb.append(str);
        return sb.toString();
    }

    // ---- 6 ----
    /** Returns true iff year is a leap year (Gregorian rule). */
    public static boolean judgeLeapYear(int year) {
        if (year % 400 == 0) {
            return true;
        }
        if (year % 100 == 0) {
            return false;
        }
        return year % 4 == 0;
    }

    // ---- 7 ----
    /**
     * Returns the number of days in the given month (1-12) of the given
     * year. Returns -1 if month is out of range. February uses leap-year
     * rules.
     */
    public static int calcDaysInMonth(int year, int month) {
        if (month < 1 || month > 12) {
            return -1;
        }
        switch (month) {
            case 1:
            case 3:
            case 5:
            case 7:
            case 8:
            case 10:
            case 12:
                return 31;
            case 4:
```

```
            case 6:
            case 9:
            case 11:
                return 30;
            case 2:
                return judgeLeapYear(year) ? 29 : 28;
            default:
                return -1;
        }
    }

    // ---- 8 ----
    /** Returns the calendar quarter (1-4) for month (1-12), or -1 if invalid. */
    public static int getQuarter(int month) {
        if (month < 1 || month > 12) {
            return -1;
        }
        return (month - 1) / 3 + 1;
    }

    // ---- 9 ----
    /**
     * Converts a 3-char month abbreviation (case-sensitive, e.g. "Jan",
     * "Sep") into its month number (1-12). Returns -1 on null, wrong
     * length, or unknown abbreviation.
     */
    public static int monAbbr2month(String abrr) {
        if (abrr == null || abrr.length() != 3) {
            return -1;
        }
        switch (abrr) {
            case "Jan":
                return 1;
            case "Feb":
                return 2;
            case "Mar":
                return 3;
            case "Apr":
                return 4;
            case "May":
                return 5;
            case "Jun":
                return 6;
            case "Jul":
                return 7;
            case "Aug":
                return 8;
            case "Sep":
                return 9;
            case "Oct":
                return 10;
            case "Nov":
                return 11;
            case "Dec":
                return 12;
            default:
                return -1;
        }
    }

    // ---- 10 ----
    /**
     * Inverse of monAbbr2month: returns "Jan".."Dec" for month 1..12, and
     * "(invalid)" otherwise.
```

```
     */
    public static String month2MonAbbr(int month) {
        switch (month) {
            case 1:
                return "Jan";
            case 2:
                return "Feb";
            case 3:
                return "Mar";
            case 4:
                return "Apr";
            case 5:
                return "May";
            case 6:
                return "Jun";
            case 7:
                return "Jul";
            case 8:
                return "Aug";
            case 9:
                return "Sep";
            case 10:
                return "Oct";
            case 11:
                return "Nov";
            case 12:
                return "Dec";
            default:
                return "(invalid)";
        }
    }
}


=== Test failures observed for that implementation ===

1 of 62 tests failed.
 - test51(comp5111.assignment.cut.NewSubject_ESTest) :: arrays first differed at element
[0]; expected:<6> but was:<316>

Produce a corrected NewSubject.java that fixes ALL of the failures above without breaking
previously passing tests.
```

Raw LLM response (8839 chars)

```
```java
package comp5111.assignment.cut;

/**
 * Task 4 reduced CUT.
 *
 * Ten public-static methods selected from {@link Subject}, with their
 * documented contracts and one private helper (isOneOf).  The patched
 * (correct) implementations from Task 2 are used so that EvoSuite's
 * generated tests pin down the *intended* behaviour, against which the
 * LLM's regenerated code is judged.
 *
 * Every signature is identical to the corresponding method in Subject.
 */
public class NewSubject {
```
```

```
    // ---- 1 ----
    /**
     * Returns true if str starts with prefix ignoring case, else false.
     * Returns false on any null argument or if str is shorter than prefix.
     */
    public static boolean startsWithIgnoreCase(String str, String prefix) {
        if (str == null || prefix == null) {
            return false;
        }
        if (str.length() < prefix.length()) {
            return false;
        }
        for (int i = 0; i < prefix.length(); i++) {
            char c1 = str.charAt(i);
            char c2 = prefix.charAt(i);
            if (Character.toLowerCase(c1) != Character.toLowerCase(c2)) {
                return false;
            }
        }
        return true;
    }

    // ---- 2 ----
    /**
     * Returns the prefix of str up to (but not including) the first
     * character that appears in {@code terminators}.  If no terminator
     * is found, the whole string is returned.
     */
    public static String parseToken(final String str, final char[] terminators) {
        if (str == null) {
            return null;
        }
        if (terminators == null || terminators.length == 0) {
            return str;
        }
        int len = str.length();
        for (int i = 0; i < len; i++) {
            char ch = str.charAt(i);
            if (isOneOf(ch, terminators)) {
                return str.substring(0, i);
            }
        }
        return str;
    }

    private static boolean isOneOf(char ch, final char[] charray) {
        if (charray == null) {
            return false;
        }
        for (char c : charray) {
            if (c == ch) {
                return true;
            }
        }
        return false;
    }

    // ---- 3 ----
    /**
     * Returns the LAST contiguous run of digits in str interpreted as
     * an int. Empty/null/no-digit inputs return 0.
     * Examples: "1234" -> 1234, "a" -> 0, "1234a123" -> 123.
     */
    public static int extractIntInStr(String str) {
```

```
        if (str == null || str.isEmpty()) {
            return 0;
        }
        int end = -1;
        int start = -1;
        for (int i = str.length() - 1; i >= 0; i--) {
            char c = str.charAt(i);
            if (Character.isDigit(c)) {
                if (end == -1) {
                    end = i;
                }
                start = i;
            } else if (end != -1) {
                break;
            }
        }
        if (end == -1) {
            return 0;
        }
        int result = 0;
        for (int i = start; i <= end; i++) {
            result = result * 10 + (str.charAt(i) - '0');
        }
        return result;
    }

    // ---- 4 ----
    /**
     * Parses a version string of form a, a.b, a.b.c, or a.b.c.d into a 4-int
     * array. Trailing components default to 0. More than 4 dot-segments or
     * malformed (consecutive dots) produces null.
     */
    public static int[] getVersionNo(final String versionString) {
        if (versionString == null || versionString.isEmpty()) {
            return null;
        }

        String[] parts = versionString.split("\\.", -1);
        if (parts.length == 0 || parts.length > 4) {
            return null;
        }

        int[] version = new int[] {0, 0, 0, 0};

        for (int i = 0; i < parts.length; i++) {
            String p = parts[i];
            // malformed: empty segment (leading dot, trailing dot, or "..")
            if (p.isEmpty()) {
                return null;
            }

            // extract the last contiguous run of digits in this segment
            int end = -1;
            int start = -1;
            for (int j = p.length() - 1; j >= 0; j--) {
                char c = p.charAt(j);
                if (Character.isDigit(c)) {
                    if (end == -1) {
                        end = j;
                    }
                    start = j;
                } else if (end != -1) {
                    break;
                }
```

```
            }
            if (end == -1) {
                // no digits at all => malformed
                return null;
            }

            int num = 0;
            for (int j = start; j <= end; j++) {
                num = num * 10 + (p.charAt(j) - '0');
            }

            // place directly into the corresponding slot; trailing components remain 0
            version[i] = num;
        }

        return version;
    }

    // ---- 5 ----
    /**
     * Pads str on the left with padChar so the resulting length is at least
     * length. Null str is treated as "". If str is already long enough it is
     * returned unchanged.
     */
    public static String padLeft(String str, short length, char padChar) {
        if (str == null) {
            str = "";
        }
        int len = str.length();
        if (length <= len) {
            return str;
        }
        int pads = length - len;
        StringBuilder sb = new StringBuilder(length);
        for (int i = 0; i < pads; i++) {
            sb.append(padChar);
        }
        sb.append(str);
        return sb.toString();
    }

    // ---- 6 ----
    /** Returns true iff year is a leap year (Gregorian rule). */
    public static boolean judgeLeapYear(int year) {
        if (year % 400 == 0) {
            return true;
        }
        if (year % 100 == 0) {
            return false;
        }
        return year % 4 == 0;
    }

    // ---- 7 ----
    /**
     * Returns the number of days in the given month (1-12) of the given
     * year. Returns -1 if month is out of range. February uses leap-year
     * rules.
     */
    public static int calcDaysInMonth(int year, int month) {
        if (month < 1 || month > 12) {
            return -1;
        }
        switch (month) {
```

```
            case 1:
            case 3:
            case 5:
            case 7:
            case 8:
            case 10:
            case 12:
                return 31;
            case 4:
            case 6:
            case 9:
            case 11:
                return 30;
            case 2:
                return judgeLeapYear(year) ? 29 : 28;
            default:
                return -1;
        }
    }

    // ---- 8 ----
    /** Returns the calendar quarter (1-4) for month (1-12), or -1 if invalid. */
    public static int getQuarter(int month) {
        if (month < 1 || month > 12) {
            return -1;
        }
        return (month - 1) / 3 + 1;
    }

    // ---- 9 ----
    /**
     * Converts a 3-char month abbreviation (case-sensitive, e.g. "Jan",
     * "Sep") into its month number (1-12). Returns -1 on null, wrong
     * length, or unknown abbreviation.
     */
    public static int monAbbr2month(String abrr) {
        if (abrr == null || abrr.length() != 3) {
            return -1;
        }
        switch (abrr) {
            case "Jan":
                return 1;
            case "Feb":
                return 2;
            case "Mar":
                return 3;
            case "Apr":
                return 4;
            case "May":
                return 5;
            case "Jun":
                return 6;
            case "Jul":
                return 7;
            case "Aug":
                return 8;
            case "Sep":
                return 9;
            case "Oct":
                return 10;
            case "Nov":
                return 11;
            case "Dec":
                return 12;
```

```
            default:
                return -1;
        }
    }

    // ---- 10 ----
    /**
     * Inverse of monAbbr2month: returns "Jan".."Dec" for month 1..12, and
     * "(invalid)" otherwise.
     */
    public static String month2MonAbbr(int month) {
        switch (month) {
            case 1:
                return "Jan";
            case 2:
                return "Feb";
            case 3:
                return "Mar";
            case 4:
                return "Apr";
            case 5:
                return "May";
            case 6:
                return "Jun";
            case 7:
                return "Jul";
            case 8:
                return "Aug";
            case 9:
                return "Sep";
            case 10:
                return "Oct";
            case 11:
                return "Nov";
            case 12:
                return "Dec";
            default:
                return "(invalid)";
        }
    }
}
```

Extracted NewSubject.java for this round

```java
package comp5111.assignment.cut;

/**
 * Task 4 reduced CUT.
 *
 * Ten public-static methods selected from {@link Subject}, with their
 * documented contracts and one private helper (isOneOf).  The patched
 * (correct) implementations from Task 2 are used so that EvoSuite's
 * generated tests pin down the *intended* behaviour, against which the
 * LLM's regenerated code is judged.
 *
 * Every signature is identical to the corresponding method in Subject.
 */
public class NewSubject {

    // ---- 1 ----
    /**
```

```
     * Returns true if str starts with prefix ignoring case, else false.
     * Returns false on any null argument or if str is shorter than prefix.
     */
    public static boolean startsWithIgnoreCase(String str, String prefix) {
        if (str == null || prefix == null) {
            return false;
        }
        if (str.length() < prefix.length()) {
            return false;
        }
        for (int i = 0; i < prefix.length(); i++) {
            char c1 = str.charAt(i);
            char c2 = prefix.charAt(i);
            if (Character.toLowerCase(c1) != Character.toLowerCase(c2)) {
                return false;
            }
        }
        return true;
    }

    // ---- 2 ----
    /**
     * Returns the prefix of str up to (but not including) the first
     * character that appears in {@code terminators}.  If no terminator
     * is found, the whole string is returned.
     */
    public static String parseToken(final String str, final char[] terminators) {
        if (str == null) {
            return null;
        }
        if (terminators == null || terminators.length == 0) {
            return str;
        }
        int len = str.length();
        for (int i = 0; i < len; i++) {
            char ch = str.charAt(i);
            if (isOneOf(ch, terminators)) {
                return str.substring(0, i);
            }
        }
        return str;
    }

    private static boolean isOneOf(char ch, final char[] charray) {
        if (charray == null) {
            return false;
        }
        for (char c : charray) {
            if (c == ch) {
                return true;
            }
        }
        return false;
    }

    // ---- 3 ----
    /**
     * Returns the LAST contiguous run of digits in str interpreted as
     * an int. Empty/null/no-digit inputs return 0.
     * Examples: "1234" -> 1234, "a" -> 0, "1234a123" -> 123.
     */
    public static int extractIntInStr(String str) {
        if (str == null || str.isEmpty()) {
            return 0;
```

```
        }
        int end = -1;
        int start = -1;
        for (int i = str.length() - 1; i >= 0; i--) {
            char c = str.charAt(i);
            if (Character.isDigit(c)) {
                if (end == -1) {
                    end = i;
                }
                start = i;
            } else if (end != -1) {
                break;
            }
        }
        if (end == -1) {
            return 0;
        }
        int result = 0;
        for (int i = start; i <= end; i++) {
            result = result * 10 + (str.charAt(i) - '0');
        }
        return result;
    }

    // ---- 4 ----
    /**
     * Parses a version string of form a, a.b, a.b.c, or a.b.c.d into a 4-int
     * array. Trailing components default to 0. More than 4 dot-segments or
     * malformed (consecutive dots) produces null.
     */
    public static int[] getVersionNo(final String versionString) {
        if (versionString == null || versionString.isEmpty()) {
            return null;
        }

        String[] parts = versionString.split("\\.", -1);
        if (parts.length == 0 || parts.length > 4) {
            return null;
        }

        int[] version = new int[] {0, 0, 0, 0};

        for (int i = 0; i < parts.length; i++) {
            String p = parts[i];
            // malformed: empty segment (leading dot, trailing dot, or "..")
            if (p.isEmpty()) {
                return null;
            }

            // extract the last contiguous run of digits in this segment
            int end = -1;
            int start = -1;
            for (int j = p.length() - 1; j >= 0; j--) {
                char c = p.charAt(j);
                if (Character.isDigit(c)) {
                    if (end == -1) {
                        end = j;
                    }
                    start = j;
                } else if (end != -1) {
                    break;
                }
            }
            if (end == -1) {
```

```
                // no digits at all => malformed
                return null;
            }

            int num = 0;
            for (int j = start; j <= end; j++) {
                num = num * 10 + (p.charAt(j) - '0');
            }

            // place directly into the corresponding slot; trailing components remain 0
            version[i] = num;
        }

        return version;
    }

    // ---- 5 ----
    /**
     * Pads str on the left with padChar so the resulting length is at least
     * length. Null str is treated as "". If str is already long enough it is
     * returned unchanged.
     */
    public static String padLeft(String str, short length, char padChar) {
        if (str == null) {
            str = "";
        }
        int len = str.length();
        if (length <= len) {
            return str;
        }
        int pads = length - len;
        StringBuilder sb = new StringBuilder(length);
        for (int i = 0; i < pads; i++) {
            sb.append(padChar);
        }
        sb.append(str);
        return sb.toString();
    }

    // ---- 6 ----
    /** Returns true iff year is a leap year (Gregorian rule). */
    public static boolean judgeLeapYear(int year) {
        if (year % 400 == 0) {
            return true;
        }
        if (year % 100 == 0) {
            return false;
        }
        return year % 4 == 0;
    }

    // ---- 7 ----
    /**
     * Returns the number of days in the given month (1-12) of the given
     * year. Returns -1 if month is out of range. February uses leap-year
     * rules.
     */
    public static int calcDaysInMonth(int year, int month) {
        if (month < 1 || month > 12) {
            return -1;
        }
        switch (month) {
            case 1:
            case 3:
```

```
            case 5:
            case 7:
            case 8:
            case 10:
            case 12:
                return 31;
            case 4:
            case 6:
            case 9:
            case 11:
                return 30;
            case 2:
                return judgeLeapYear(year) ? 29 : 28;
            default:
                return -1;
        }
    }

    // ---- 8 ----
    /** Returns the calendar quarter (1-4) for month (1-12), or -1 if invalid. */
    public static int getQuarter(int month) {
        if (month < 1 || month > 12) {
            return -1;
        }
        return (month - 1) / 3 + 1;
    }

    // ---- 9 ----
    /**
     * Converts a 3-char month abbreviation (case-sensitive, e.g. "Jan",
     * "Sep") into its month number (1-12). Returns -1 on null, wrong
     * length, or unknown abbreviation.
     */
    public static int monAbbr2month(String abrr) {
        if (abrr == null || abrr.length() != 3) {
            return -1;
        }
        switch (abrr) {
            case "Jan":
                return 1;
            case "Feb":
                return 2;
            case "Mar":
                return 3;
            case "Apr":
                return 4;
            case "May":
                return 5;
            case "Jun":
                return 6;
            case "Jul":
                return 7;
            case "Aug":
                return 8;
            case "Sep":
                return 9;
            case "Oct":
                return 10;
            case "Nov":
                return 11;
            case "Dec":
                return 12;
            default:
                return -1;
```

```
        }
    }

    // ---- 10 ----
    /**
     * Inverse of monAbbr2month: returns "Jan".."Dec" for month 1..12, and
     * "(invalid)" otherwise.
     */
    public static String month2MonAbbr(int month) {
        switch (month) {
            case 1:
                return "Jan";
            case 2:
                return "Feb";
            case 3:
                return "Mar";
            case 4:
                return "Apr";
            case 5:
                return "May";
            case 6:
                return "Jun";
            case 7:
                return "Jul";
            case 8:
                return "Aug";
            case 9:
                return "Sep";
            case 10:
                return "Oct";
            case 11:
                return "Nov";
            case 12:
                return "Dec";
            default:
                return "(invalid)";
        }
    }
}
```
